



Bài 2 Cấu trúc dữ liệu và giải thuật 2

Công nghệ thông tin (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

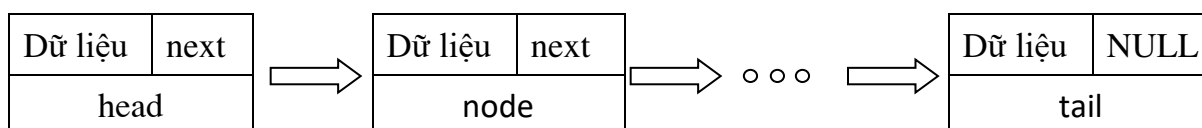
Chương 2.

DANH SÁCH LIÊN KẾT ĐÔI, LIÊN KẾT VÒNG

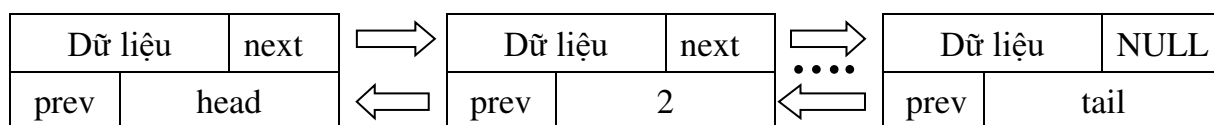
2.1. Danh sách liên kết đôi

2.1.1. Khái niệm

Trước đây ta đã tìm hiểu danh sách liên kết đơn, có sơ đồ :



Bây giờ nếu mỗi nút của danh sách có thêm một liên kết ở phía trước, đặt là prev (previos- phía trước), thì sẽ có hình ảnh của danh sách liên kết đôi :



Như vậy, danh sách liên kết đôi là một dạng mới của danh sách liên kết, trong đó mỗi nút có 2 phía liên kết, các tác động qua các nút có thể được thực hiện theo hai chiều về trước và về sau.

Mỗi nút của danh sách liên kết đôi có 3 thành phần : con trỏ prev liên kết với nút trước, dữ liệu, con trỏ next liên kết với nút sau.

Một danh sách liên kết đôi sẽ liên kết từ nút đầu tiên đến nút cuối cùng, nút đầu tiên con trỏ prev=Head, nút cuối cùng con trỏ next = NULL.

Các thao tác với danh sách liên kết đôi cũng giống như với danh sách liên kết đơn :

- ❖ Thêm một phần tử vào vị trí đầu/cuối của Danh sách liên kết đôi.
- ❖ Xóa một phần tử tại vị trí đầu/cuối của Danh sách liên kết.
- ❖ xóa một phần tử trong danh sách liên kết theo dữ liệu hay vị trí của nút.
- ❖ Hiện thị danh sách liên kết theo chiều về phía sau/trước.

2.1.2. Khai báo và khởi tạo danh sách liên kết đôi

a) Khai báo cấu trúc nút

```
struct Node
{
    int du_lieu;
    Node* prev; //liên kết với nút trước
    Node* next; //liên kết với nút sau
};
```

b) Khai báo nút đầu, nút đuôi của danh sách liên kết đôi :

```
struct Node *head, *tail; // head và tail là 2 con trỏ kiểu Node
```

Giải thích : Node là tên kiểu cấu trúc, mỗi cấu trúc kiểu Node có 3 thành phần :

a) dữ liệu

b) con trỏ prev có kiểu Node

c) con trỏ next có kiểu Node

Cách khai báo này có dạng đệ quy, mỗi con trỏ prev và next cũng sẽ có 3 a,b,c như trên

2.1.3. Tạo một nút mới để nhận dữ liệu

Đặt tên nút mới là newNode, vì nó chưa có liên kết gì nên prev và next đều gán = NULL, du_lieu0 là dữ liệu được đưa vào.

```
struct Node* Taonutmoi(int du_lieu0)
{ struct Node* newnode = new Node;//cấp bộ nhớ kiểu Node cho nút newnode
  newnode->du_lieu = du_lieu0;
  newnode->prev = NULL;
  newnode->next = NULL;
  return newnode;
}
```

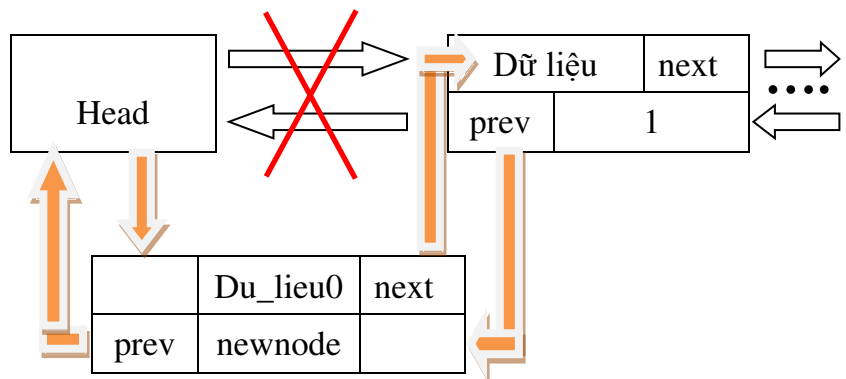
2.1.4. Thêm nút vào danh sách liên kết đôi

Thêm nút vào đầu hay cuối danh sách liên kết đôi cũng giống như thêm vào đầu/cuối danh sách liên kết đơn.

a) Thêm vào đầu

- Nếu head == NULL, sẽ cho cả head và tail = newnode.
- Nếu head != NULL, sẽ cập nhật lại head mới là newnode. Ta cần tạo liên kết giữa head hiện tại với newnode trước khi cho newnode làm head mới.

Ta có sơ đồ cơ chế thêm nút mới vào đầu danh sách liên kết đôi :

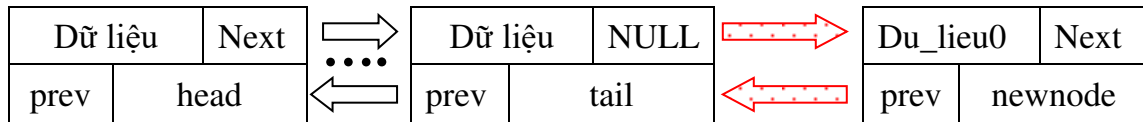


```
void ThemDau(int du_lieu0)
{ struct Node* newnode = Taonutmoi(int du_lieu0) ;
  if(head == NULL)
  { head = newnode;
    tail = newnode;
    return ;
  }
  head->prev = newnode;
  newnode->next = head;
  head = newnode;
}
```

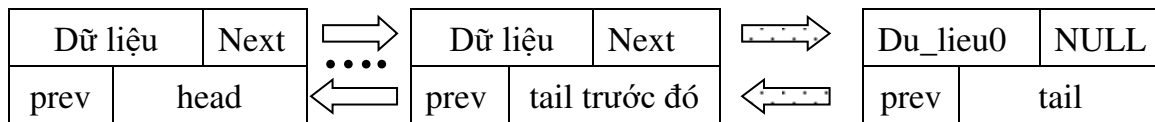
b) Thêm vào cuối

- Nếu head == NULL, newnode sẽ là head và tail .
- Nếu head != NULL, cập nhật lại tail mới là newnode. Ta cần tạo liên kết tail hiện tại với newnode trước khi để newnode làm tail mới.

Ta có sơ đồ thêm nút vào cuối danh sách liên kết đôi :



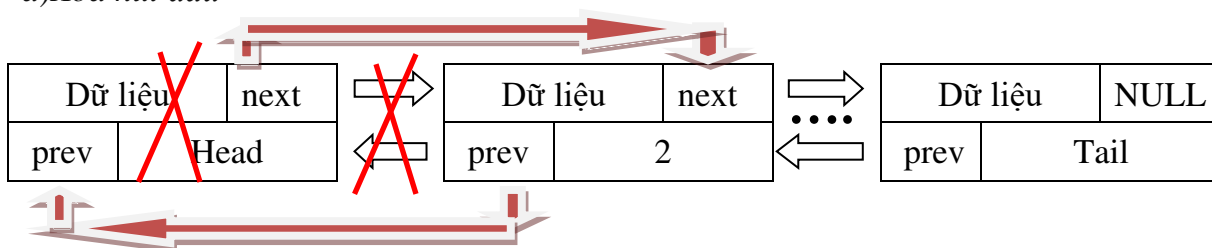
Kết quả sau khi thêm vào cuối :



```
void ThemCuoi(int du_lieu0) {
    struct Node* newnode = Taonutmoi(du_lieu0) ; //Tạo nút mới có dữ liệu 0
    if(head == NULL) { head = newnode;
        tail = newnode;
        return;
    }
    tail->next = newnode;
    newnode->prev = tail;
    tail = newnode;
}
```

2.1.5. Xóa nút

a) Xóa nút đầu

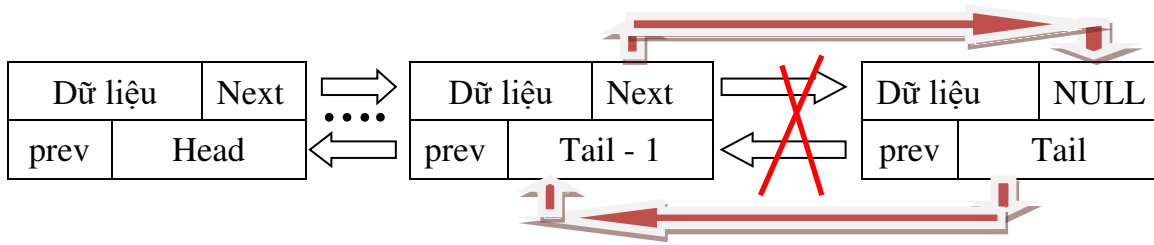


- Nếu head == NULL, chẳng có gì để xóa, return.
- Nếu head != NULL, cho head mới bằng phần tử tiếp theo và sửa prev của nó = NULL (ngắt liên kết với head cũ).

Về ý nghĩa vật lý trong bộ nhớ lưu trữ thì sau khi xóa nút đầu, nút kế tiếp đã đóng vai trò nút đầu mới, nút đầu cũ dù còn trong bộ nhớ nhưng không được quan tâm đến nữa.

```
void XoaDau() {
    if(head == NULL) { return; }
    head->prev = NULL; head = head->next;
}
```

b) Xóa nút cuối



- Nếu head == NULL, chẳng có gì để xóa
- Nếu head != NULL, cho tail mới bằng phần tử trước nó và sửa next của tail = NULL (ngắt liên kết với tail cũ).

```
void XoaCuoi() {
    if(head == NULL) { return; }
    tail = tail->prev;
    tail->next = NULL;
}
```

2.1.6. Hiện danh sách liên kết đôi

a) Hiện từ đầu tới cuối

Duyệt bắt đầu từ head cho tới trước khi gặp nút NULL bằng cách dùng con trỏ next.

```
void Hien1() {
    struct Node* p = head;
    cout<<"\n Hien tu Dau => Cuoi : ";
    while(p != NULL) { cout<<p->du_lieu; //hiện dữ liệu của nút p
                        p = p->next; //chuyển sang nút kế sau
                    }
    cout<<"\n"; system("pause");
}
```

b) Hiện từ cuối về đầu

Duyệt bắt đầu từ nút tail rồi lùi về phía trước cho đến khi gặp NULL bằng cách dùng con trỏ prev.

```
void Hien2() //Hiện từ Cuối về Đầu
{ struct Node* p = tail;
  if(p == NULL) return; // Danh sách rỗng, thoát ra
  cout<<"\n Hien tu Cuoi => Đầu : ";
  while(p != NULL) { cout<<p->du_lieu; //Hiện dữ liệu
                    p = p->prev; //chuyển lên nút phía trước
                }
  cout<<"\n"; system("pause");
}
```

2.1.7. Tìm nút theo dữ liệu trong danh sách liên kết đôi

```
bool TimDulieu( int a ){
    Node *p = head;
    while(p != NULL) {
        if(p->du_lieu == a ) return true;
        else p = p->next;
    }
    return false;
}
```

Trong hàm main(), ta viết các dòng lệnh :

```
cout<<"\nNhập dữ liệu nút cần tìm trong danh sách: "; cin>> a;
if(TimDulieu(node,a) == true){
    cout<<"Đã tìm thấy dữ liệu " <<k<<" \n";
}
else{
    cout<<"không tìm thấy ";
}
```

Bài tập : (chú ý : trong bài này khi nói danh sách được hiểu là danh sách liên kết đôi)

1. Vẽ sơ đồ khối thuật toán cho các hàm trong danh sách liên kết đôi.

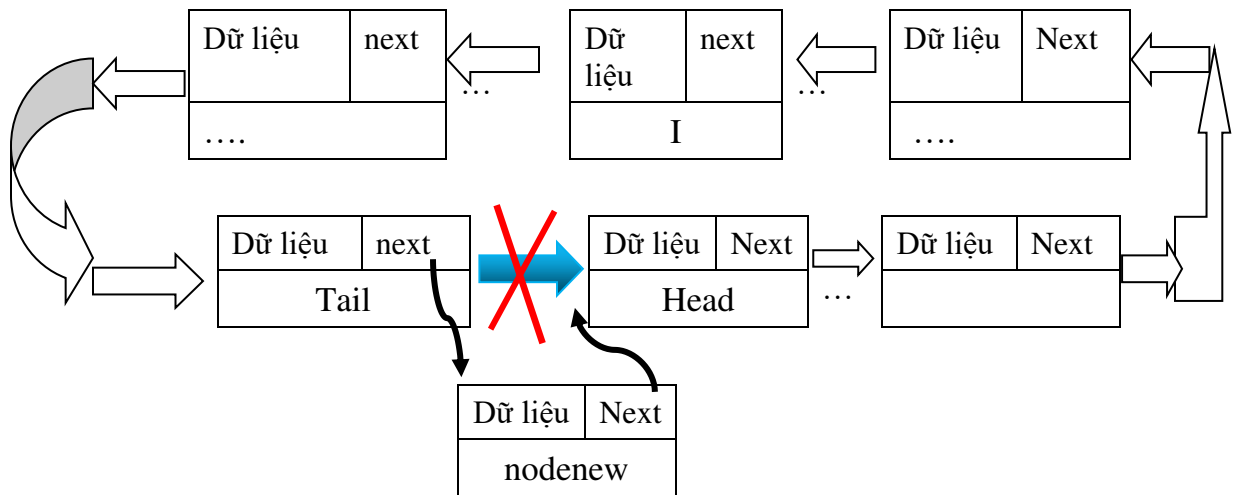
2. Ghép nối các khai báo, các hàm và viết thêm hàm main() có các chức năng :

- Khởi tạo danh sách rỗng ban đầu
- Thêm các nút vào đầu, cuối
- Hiện danh sách
- Xóa nút đầu/cuối
- Hiện lại danh sách sau khi xóa một nút

2.2.4. Thêm nút vào danh sách liên kết vòng

a) Thêm vào đầu

```
Node * ThemDau(Node * tail, int du_lieu)
{ Node * nodenew = TaoNode(du_lieu); //tạo nút mới, có dữ liệu
  if (tail == NULL) { tail = nodenew;
                      nodenew -> next = nodenew; }
  else { nodenew -> next = tail -> next;
         tail -> next = nodenew;
         tail = nodenew; }
  return tail;
}
```



b) Thêm vào cuối . Trong liên kết vòng đơn, thêm vào cuối cũng giống như thêm vào đầu.

```
Node * ThemCuoi(Node * tail, int du_lieu)
{ return ThemDau(tail, du_lieu) -> next; }
```

c) Thêm nút vào một vị trí k bất kỳ đã xác định

Đặt n là số phần tử của danh sách liên kết vòng đơn, *k* là số đếm trong khoảng từ 1 đến n.

```
Node * Themk(Node * tail, int du_lieu, int k) // Thêm vào vị trí k bất kỳ
{ int n = Count(tail), i;
  if (k < 1 || k > n + 1) { cout<<"\n  Không có vị trí này để thêm \n"; }
  else
  { if (tail == NULL) return ThemDau(tail, du_lieu);
    Node * node = TaoNode(du_lieu); Node* nodeht = tail; //nodeht – nút hiện thời
    for (i = 1; i != k; i++) nodeht = nodeht -> next; //khi i==k thì ra khỏi for
    node -> next = nodeht -> next;
    nodeht -> next = node;
    if (k == n + 1) tail = node;
  }
  return tail;
}
```


2.2.5.Xóa nút trong danh sách liên kết vòng đơn

a)Xóa nút theo giá trị đã chỉ định

Ý tưởng thuật toán của hàm xóa nút có giá trị bằng giá trị đã cho trong danh sách liên kết vòng đơn như sau :

- Trong thân hàm cần xác định biến tail của danh sách và du_lieu dữ liệu cần xóa.
- Tạo một biến con trỏ nodeht – nút hiện thời – để nhận tail
- Tạo một biến con trỏ nodetrc – nút trước nút hiện thời
- Nếu tail ==NULL là danh sách rỗng thì return tail, không có gì để xóa, còn nếu danh sách chỉ có 1 nút, tức là đầu và đuôi giống nhau, tail==tail->next, và nếu có dữ liệu == du_lieu thì sẽ gán tail=NULL và xóa nodeht nếu không thì trả về tail;
- Trường hợp danh sách có nhiều hơn 1 nút thì tổ chức vòng lặp thực hiện lần lượt các bước :
 - + gán nodetrc = nodeht; (*nodeht là tail*)
 - + chuyển nodeht sang nút kế tiếp : nodeht = nodeht->next (*bây giờ nodeht là head*)
 - +Nếu du_lieu của nodeht bằng du_lieu cần xóa thì xóa nodeht, để xóa nodeht cần thực hiện:
 - Gán nodetrc->next = nodeht->next
 - Nếu nodeht==tail thì tail=nodetrc
 - Xóa nodeht
 - Đặt lại nodeht = nodetrc->next
 - + Kết thúc vòng lặp while (*nodeht != tail*);

Ta có hàm như sau :

```
Node * XoaDulieu(Node * tail, int du_lieu) {
    Node * nodeht = tail, * nodetrc; // nodeht – nút hiện thời, nodetrc – nút trước đó
    if (tail == NULL) return tail;
    else
        if (tail == tail -> next) {
            if (tail -> du_lieu == du_lieu) { tail = NULL; delete nodeht; }
            return tail;
        }
    do {
        nodetrc = nodeht;
        nodeht = nodeht -> next; // nodeht chính là head
        if (nodeht -> du_lieu == du_lieu)
            { nodeht -> next = nodetrc -> next;
              if (nodeht == tail) tail = nodetrc;
              delete nodeht;
              nodeht = nodetrc -> next;
            }
    } while (nodeht != tail);
    return tail;
}
```

b)Xóa theo vị trí chỉ định

```
Node * XoaVitri(Node * tail, int vitri) {
    Node * nodeht, * nodetrc = tail; // nodeht – nút hiện thời, nodetrc – nút trước đó
    int n = Count(tail), i;
```

```

if (vitri < 1 || vitri > n) {
    cout<< " Khong co vi tri de xoa \n";
} else if (n == 1) {
    tail = NULL;
    delete nodeht; // nếu chỉ có 1 nút thì xóa luôn nút hiện thời
} else {
    nodeht = tail -> next; // nodeht chính là head
    for (i = 1; i < vitri; i++) { //vòng for di chuyển tìm đến vị trí đã cho
        nodetrc = nodeht;
        nodeht = nodeht -> next;
    } // Ra khỏi vòng for
    nodetrc -> next = nodeht -> next; //liên kết của nút trước đó lấy liên kết của nút hiện thời
    if (nodeht == tail) tail = nodetrc;
    delete nodeht; // xóa nút hiện thời
}
return tail;
}

```

Bài tập

1. Vẽ sơ đồ khối thuật toán cho các hàm trong danh sách liên kết vòng.
2. Vận dụng các khai báo cấu trúc, biến và các hàm đối với danh sách liên kết vòng, viết thêm hàm main() có các chức năng :

- Khai báo và khởi tạo danh sách liên kết vòng rỗng ban đầu

Thêm vào đầu

Thêm vào cuối

Thêm vào sau vị trí đã cho

Xóa nút theo dữ liệu đã cho

Xóa nút theo vị trí

Hiện danh sách liên kết vòng